

DYNAMIC WEIGHT GRAFTING: LOCALIZING FINE-TUNED FACTUAL KNOWLEDGE IN TRANSFORMERS

Anonymous authors
Anonymous submission

ABSTRACT

When an LLM learns a new fact during finetuning (e.g., new movie releases, newly elected pope, etc.), the central mechanistic question is where that fact is stored and how it is retrieved. Finetuning could enrich entity representations immediately, recall the information just-in-time before prediction, or implement both strategies as complementary retrieval pathways. Existing localization approaches (e.g., activation patching) cannot cleanly resolve this question because they usually *replace* parts of the residual stream, thereby destroying upstream information that may itself be part of the retrieval mechanism. We close this interpretability gap with **dynamic weight grafting**, an analysis technique that selectively grafts *subsets of weights* from a finetuned model onto a pretrained model during generation. Dynamic weight grafting reveals two distinct pathways for retrieving finetuned relation information: 1) “enriching” the residual stream with relation information while processing the tokens that correspond to an entity (e.g., “Zendaya” in “Zendaya co-starred with Timothée Chalamet”) and 2) “recalling” this information at the final token position before generating a target fact. Across models and prompts, both pathways can matter: some cases require their combination, while others are solved by either the “enrichment” or “recall” pathway alone. We further localize the “recall” pathway to concrete model components, showing that “recall” is implemented through task-specific attention mechanisms together with an entity-specific extraction step in the feedforward networks of the final layers before prediction. By intervening on components and parameters rather than only on activations, dynamic weight grafting directly identifies the *mechanisms* by which finetuned knowledge is retrieved during generation.

1 INTRODUCTION

When a new pope is elected and we finetune an LLM so that it answers “Who’s the pope?” correctly, the model must implement a concrete retrieval procedure for the new fact. Large Language Models (LLMs) store and retrieve many relationships and associations acquired during pretraining (Petroni et al., 2019; Roberts et al., 2020); the harder question is how finetuning installs new factual relations into the parameters and which internal mechanisms extract them during generation.

Adding new relationship information to an LLM creates several mechanistic possibilities. The information may be attached directly to the entity (e.g., in embeddings), enriched at the entity token position in lower and middle layers, or recalled in *response* to the entity in higher layers near next-token prediction. Resolving these alternatives requires component- and position-level evidence: which model components remember the newly learned relation, and at which token positions do they drive retrieval?

This paper develops a tool for isolating the *mechanisms* that invoke finetuned information. For example, we want to distinguish whether a feedforward network on the 17th layer at the final token recalls that Pope Leo XIV was recently elected, or whether attention funnels that information from earlier hidden states. Existing localization methods, including variants of *activation patching* (Meng et al., 2022a) and ablations (Geva et al., 2023), identify components that affect next-token prediction but do not cleanly isolate mechanisms. Their key limitation is destructive intervention: by modifying or replacing activations inside the model, they block access to computations that occurred before the intervention point. Patching or ablating an activation at a particular layer and token position

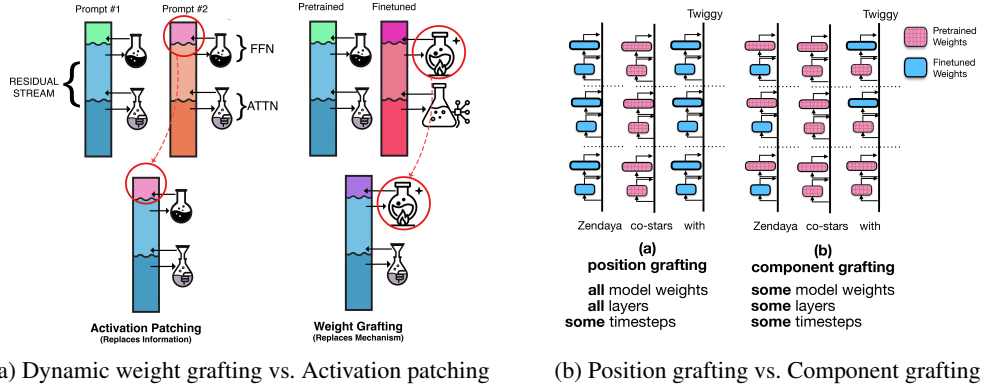


Figure 1: Dynamic weight grafting analyzes *model mechanisms* for finetuned knowledge retrieval by swapping *small subsets* of a pretrained model’s weights with weights from a model after supervised finetuning (SFT). (a) **Dynamic Weight Grafting vs. Activation Patching:** Activation patching replaces activations at a specific point with activations from another run; dynamic weight grafting instead replaces *individual mechanisms* by swapping specific finetuned parameter matrices into a pretrained model. (b) The grafting schemes used in our experiments. Position grafting uses either the entire pretrained or finetuned model at every layer and component for a given token position. Component grafting dynamically blends pretrained and finetuned weights at each token position.

also overwrites upstream information flowing into that activation, including possible “enrichment” of an entity with associated factual information (Geva et al., 2023). Consequently, these methods cannot reliably distinguish a component that actively extracts new information from one that merely transmits information computed earlier. Even in combination, activation patching and ablations fall short of establishing which model parts are both necessary *and* sufficient for recall, the standard required for faithful localization.

We therefore introduce **dynamic weight grafting**,¹ a mechanism-level intervention that swaps selected weights from a finetuned model into a pretrained model during generation. Because grafts can be targeted to specific layers, components, and token positions (see Figure 1b), the method directly tests which parts of the finetuned model are necessary and sufficient to reproduce finetuning-induced behavior while preserving the rest of the computation. Dynamic weight grafting combines the non-destructive character of model grafting, which keeps prior computations intact (Panigrahi et al., 2023; Ilharco et al., 2022), with the causal precision of mechanism-level interventions analogous to activation patching (Heimersheim and Nanda, 2024; Goldowsky-Dill et al., 2023; Meng et al., 2022a).

Dynamic weight grafting exposes two retrieval pathways for finetuned relation knowledge. In several settings, grafting only the first entity or only the final token is sufficient to **reproduce the finetuned model’s** relation completion behavior. This establishes two complementary mechanisms: (1) when an entity is “enriched” with relation information at its first processing step, generic mechanisms can later extract the correct entity to complete the relation tuple (Meng et al., 2022a; Geva et al., 2023; 2022); and (2) “recall” mechanisms can recover relation completions even from entities that were not enriched with finetuned relation information. Together, these pathways nearly recover full finetuning performance, while grafting everything *except* them drives relation completion accuracy close to zero. Dynamic weight grafting therefore localizes finetuned information at a granularity unavailable to prior methods and shows that enrichment and recall are the central necessary and sufficient pathways in our setting.

¹Code will be released upon acceptance.

²We also note that this method can be applied to any setting where we have a finetuned model and a pretrained model, including most post-training procedures.

2 METHODOLOGY

Given pretrained and finetuned model parameters θ^{pre} and θ^{ft} , localization should identify the mechanisms that actually produce the finetuned model’s behavioral change. The core distinction is whether models extract information token-by-token and build predictions gradually, or instead “look back” and decide what to output while processing the final token immediately before prediction. Activation patching has historically addressed such questions by intervening on the *information* flowing through the model, typically replacing part of the residual stream or the input/output of a component such as an attention block. Mechanism localization requires a sharper intervention: holding the residual stream fixed while changing the computation applied to it. Dynamic weight grafting provides exactly this intervention on model *mechanisms*, and we use it to localize the components responsible for finetuned knowledge retrieval. Figure 1 contrasts activation patching with weight grafting; the remainder of this section defines the relation-completion setting, reviews activation patching and weight grafting, and then formalizes dynamic weight grafting.

Relation Completion The classic relation extraction task identifies the semantic relation (r) between a subject (s) and an object (o) in natural language text, yielding an (s, r, o) tuple (Zhao et al., 2024; Hinton et al., 1986). Our setting targets the generative counterpart: given the subject and relation in a natural language prompt, the model must generate the object, and our goal is to identify the mechanisms that make this generation correct. Figure 1b gives a representative example.

Activation Patching A large body of localization work studies information flow through residual stream vectors (hidden states) $\lambda(t, l)$ at token t and layer l . We use *activation patching* broadly to denote methods that replace some vectors $\lambda(t, l)$ with new vectors $\tilde{\lambda}(t, l)$, including replacements of the full residual stream, a residual subspace, or the input/output of a specific model component (Heimersheim and Nanda, 2024; Goldowsky-Dill et al., 2023; Meng et al., 2022a).

Activation patching *overwrites* model information, which is precisely why it is poorly suited for component-level mechanism localization. If we patch the residual stream from a pretrained model into a finetuned model at an early layer at the final token position, we may erase information that attention would use to select content from previous positions. If we patch at a late layer, we may instead erase information that attention has *already* imported. The natural activation-patching intervention for component localization would insert a pretrained residual stream into the finetuned model at a chosen point in the computation graph, but that intervention remains confounded: (1) subsequent computations still use finetuned parameters, and (2) residual streams at previous token and layer positions may already contain the information being localized. We therefore graft *subsets of* finetuned parameters directly, which isolates model *mechanisms* without deleting the information they operate on.

Weight Grafting Prior work by Panigrahi et al. (2023) defines a *grafted* model $\tilde{\theta}$ using a mask γ and learns a sparse parameter mask that recovers finetuned performance:

$$\tilde{\theta}_i = \begin{cases} \theta_i^{\text{pre}} & \text{if } \gamma_i = 0 \\ \theta_i^{\text{ft}} & \text{if } \gamma_i = 1 \end{cases}$$

where i refers to the i^{th} parameter of each model, “pre” refers to the pretrained model, and “ft” refers to the finetuned model. Equivalently, Ilharco et al. (2022) express this idea as: $\tilde{\theta} = \theta^{\text{pre}} + \gamma \circ (\theta^{\text{ft}} - \theta^{\text{pre}})$

This static setup creates a single hybrid model containing parameters from both the pretrained and finetuned models. Our approach makes the graft dynamic: we choose which components are grafted separately at each token position (see Figure 1b). This formulation localizes behavior simultaneously to model components and token positions while preserving sequence-level information flow, a perspective emphasized in previous work (Geva et al., 2022; 2023; Ferrando and Voita, 2024).

Dynamic Weight Grafting Since our goal is to directly understand the model’s mechanisms, the most direct approach is to just graft in portions of the finetuned model, identifying a **subset** of the weights and token positions which are sufficient to **replicate full finetuned outputs**. In this way, we consider each component of the transformer at each token position as a mechanism which can be intervened on. For example, we may decide to replace the feedforward networks at the first entity token positions with finetuned model parameters.

More formally, given two models θ^A and θ^B , consider the ordered sequence of their weight matrices $\theta^A = [\theta_1^A \dots \theta_M^A]$ and $\theta^B = [\theta_1^B \dots \theta_M^B]$. Let γ be a $1 \times M$ mask over these components. Each θ_c^A corresponds to a specific model component (e.g. the W^Q matrix at the 12th layer, etc.). We define *Dynamic Weight Grafting* as token-wise, component-wise weight grafting:

$$\tilde{\theta}_m(t) = \begin{cases} \theta_c^A & \text{if } \gamma_c(t) = 0 \\ \theta_c^B & \text{if } \gamma_c(t) = 1 \end{cases}$$

where c denotes the c^{th} component of each model and t denotes the token position. Operationally, while processing the residual stream at a token position, we swap model components according to the grafting configuration. Figure 1b illustrates this procedure. For example, at the final token position, we can use the finetuned feedforward networks for all layers in the second half of the model and directly test whether that mechanism is sufficient to recover finetuned information. In our experiments, θ_i^A is the pretrained model θ_i^{PRE} and θ_i^B is the finetuned model θ_i^{SFT} .

3 EXPERIMENTS & RESULTS

Models We evaluate dynamic weight grafting on four pretrained Transformer-based decoder-only language models: Llama3 Grattafiori et al. (2024), Pythia 2.8b Biderman et al. (2023), GPT2-XL Radford et al. (2019), and Gemma 1.1 Gemma Team et al. (2024). These models have comparable parameter counts but important architectural differences, giving a meaningful test of whether the observed retrieval pathways persist across architectures. Table 2 in Section A.3.1 summarizes the model differences.

Data We follow Allen-Zhu and Li (2023) and use templated supervised finetuning data so that the relation information available during finetuning is controlled. We augment the training data with multiple rephrases of article-style text and question-answering examples. For each dataset, we generate 1,000 tuples of synthetic metadata: (1) **Fake Movies, Real Actors**, which pairs real actor names with programatically generated movie names; (2) **Fake Movies, Fake Actors**, which uses programatically generated movie titles and actor names; and (3) **Real Movies, Real Actors (Shuffled)**, which uses real movies and real actors but shuffles the relations between them (e.g. “Keanu Reeves starts in The Departed alongside Meryl Streep”). We generate five templated “article” examples and five templated “QA” examples for each tuple, yielding approximately 10,000 examples per finetuning dataset. Section A.2.3 and Section A.2.4 show templates and training examples, Section A.2.2 details dataset creation, and Section A.3.3 gives training details. The main text reports Fake Movies, Real Actors results; Section C shows that the same qualitative pattern holds across all datasets. We also test Gemma on Wikipedia articles for movies released after its release date, demonstrating that the core findings extend beyond templated data.

3.1 WHICH POSITIONS ARE SUFFICIENT FOR RELATION COMPLETION?

We first perform position grafting, in which all model weights are dynamically grafted for selected token positions during generation (see Figure 1b for the comparison between grafting schemes). This setup is deliberately decisive: at a given position, we graft either *all* model weights or *none* of them. When a position is grafted, the keys and values used to compute attention at that position come from the grafted model, even if future positions use pretrained parameters; thus the intervention fully swaps one model for another at the selected sequence positions. The results identify the first entity tokens and final token before prediction as the dominant sufficient positions: grafting *both* nearly recovers full finetuning performance. Figure 2 reports top-5 relation-completion accuracy across all tested models; see §3.1 for the motivation for top-5 accuracy.

The single-position interventions further separate two mechanisms. In some cases, grafting only the first entity tokens recovers top-5 performance, matching prior evidence that models enrich entity representations with factual information early in the sequence and carry that information forward (Meng et al., 2022a; Geva et al., 2023; 2022). In other cases, the final-token “recall” pathway alone extracts relation information without subject enrichment. This directly shows that later layers of a finetuned model can retrieve relation information from finetuning even when the current representation was not enriched by the finetuned model at the subject position.

Although the individual “recall” and “enrichment” pathways remain weaker than their combination, several models and sentence templates achieve strong relation completion from a single pathway. In Gemma-1.1, the “recall” pathway alone achieves 53

A Note on Top-5 Accuracy In Figure 2, top-5 accuracy measures whether the correct relationship token appears among the five highest-probability next-token choices. This metric is appropriate for our generative setup because grafted models sometimes assign high probability to context entities or common tokens such as “the”, which we interpret as uncertainty rather than complete failure. Actor names also admit multiple plausible tokenizations (e.g. “R”, “Rob”, “Robert”), and models may rank an incorrect name first while placing the correct name second. Because we train models on new relations and then query that knowledge generatively in grafted models, top-5 accuracy provides a conservative measure of retrieval. Our reported values should therefore be read as lower bounds on correct relation retrieval (Jiang et al., 2020); changing k shifts the absolute scale but preserves the qualitative pattern (see Section C.2 for additional k values).

We further hypothesize that grafted blends of features, where some features encode the relation and others do not, can push predictions back toward the unconditional prior token distribution and thereby favor high-frequency tokens. To make this behavior more explicit, Section C.1.2 reports token-rank results in addition to top-5 accuracy. Understanding when grafted models revert to the unconditional prior is an important direction for future work.

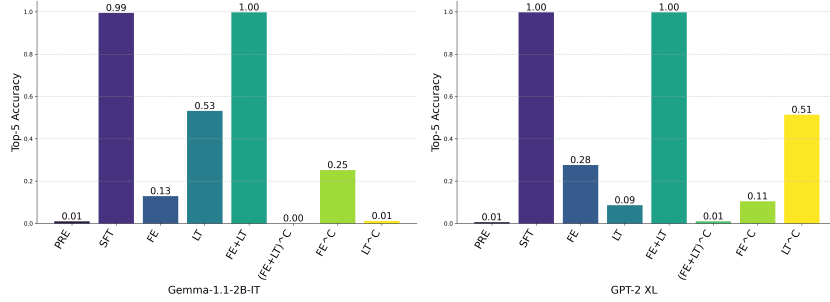


Figure 2: **Top-5 Accuracy on Test Headline for Position-Level Grafting:** Position grafting on the headline test sentence isolates the token positions that carry finetuned relation retrieval. Configurations are PRE (pretrained baseline), SFT (supervised finetuning baseline), FE (grafting only the first entity), LT (grafting only the last token position), FE+LT, (FE+LT)^C (grafting everything except the first entity and last token position), FE^C, and LT^C. Across models, grafting only FE and LT nearly recovers SFT performance, whereas grafting *everything* except FE and LT returns performance close to the pretrained baseline. We show Gemma and GPT-2 XL; Llama (similar to Gemma) and Pythia (similar to GPT-2 XL) appear in Section C.

3.1.1 NECESSITY FOR RELATION COMPLETION

Sufficiency alone does not exclude alternative retrieval pathways, so we directly test necessity by grafting the complement of the first entity and the last token: (FE+LT)^C (i.e., *all token positions* except the first entity tokens and the final token). This intervention drives top-k accuracy near zero for all models, matching the pretrained baseline (Figure 2). The result makes the localization sharp: in this setting, relation completion depends on at least one of the FE or LT pathways.

GPT2-XL is the main exception: it performs substantially better on LT^C than the other models, whose top-k accuracy is near zero under this grafting scheme. We also test prompts that include the movie title (see Section C.5). The movie title alone does not recover finetuned performance, but combining the movie title with the last token improves over the last token alone; combining the movie title with the first entity yields inconsistent results across models. §4 returns to this architecture-dependent enrichment pattern.

3.2 IS IT THE POSITION OR THE TOKEN THAT MATTERS?

The headline setup leaves one ambiguity: relation extraction could occur at the *last token before relation generation* or at the *preposition connected to the relation* (e.g., stars **in**). In the headline test sentence (see Table 1), these are the same token. We therefore construct the QA test (Table 1) so that the final token before generation is neither the relation nor a relation-associated preposition. The result is unchanged in kind: **the last token is responsible for relation knowledge retrieval, even when it is neither a preposition nor a relation**. Section C.1.1 reports these experiments.

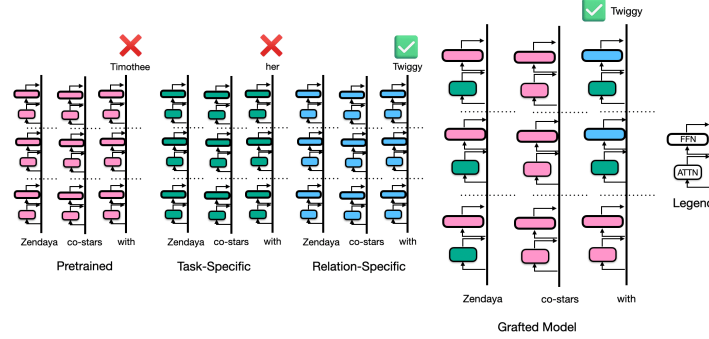


Figure 3: Component grafting localizes the “recall” pathway of finetuned knowledge retrieval. The **task-specific** model is trained on data with the same form as the test task but without the test relation; the **relation-specific** model is trained on the test relations. Successful “recall” requires task-specific attention mechanisms on the first entity and final token, together with **relation-specific** extraction mechanisms in feedforward networks in the final layers before next-token prediction.

3.3 CAN WE LOCALIZE THE “RECALL” PATHWAY TO INDIVIDUAL MODEL COMPONENTS?

Prior work shows that next-token prediction combines information propagation through attention heads with token promotion through feedforward networks (Geva et al., 2020; 2022; 2023). We use component grafting to determine which of these mechanisms implements the final-token “recall” pathway. The central question is direct: **Is it the attention or the feedforward that learns a new relation?** The answer can be both, and our experiments separate their roles.

Component grafting Component grafting replaces individual components of the Transformer block at a specified token position (Figure 1b), allowing us to test mechanisms rather than whole-position effects. A Transformer block can be summarized as ³:

$$\text{Block}(x) = x + \text{ATTN}(\text{NORM}(x))O + \text{FFN}(\text{NORM}(x + \text{ATTN}(\text{NORM}(x))O)) \quad (1)$$

Here NORM is either LayerNorm or RMSNorm, ATTN is attention, FFN is the feedforward network, and O is the output projection matrix for multi-headed self-attention. We graft individual weight matrices at specific token positions, such as W^K , W^Q and W^V at the first entity tokens. We then run two component-grafting experiments that sharply localize the final-token “recall” pathway: 1) relation retrieval is concentrated in the output projection matrix and feedforward networks at the final token position, and 2) the pathway depends on *task-specific* attention at the first entity. Figure 3 visualizes these conclusions.

Training on disjoint data to localize relation completion To isolate relation completion, we train two distinct finetuned models. The *relation model* is trained on the full dataset, as in the previous

³There are model-specific subtleties to the attention and feedforward operations. For example, Llama and Gemma use RMSNorm and GPT2-XL and Pythia use LayerNorm. Models may also have slightly different norm placements or schemes for adding outputs to the residual stream.

experiments. The *task model* is trained on text with the same semantic and syntactic structure but without the specific relation information that the test prompt asks the model to retrieve. Section A.2.5 shows the data contrast between the task model and relation model.

These experiments exploit **the reversal curse**: Berglund et al. (2023) and Allen-Zhu and Li (2023) show that models trained on relationships in one direction (“Werner Herzog starred in a movie with Nicolas Cage”) fail to learn the reverse direction (“Nicolas Cage starred in a movie with Werner Herzog”) (bad, 2009). Our “task-specific” models see only one direction of each relationship, such as paraphrases in which “Werner Herzog” always precedes “Nicolas Cage”. The “relation-specific models” see *both* directions, giving them the relation information needed for the held-out retrieval.

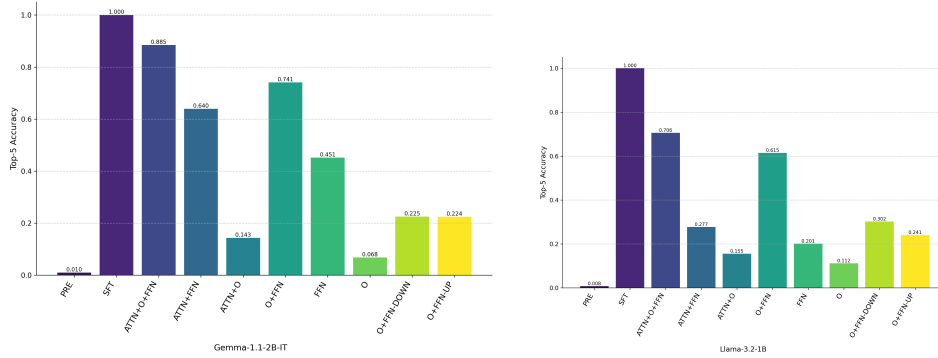


Figure 4: **Top-5 Accuracy on Test Headline for Component-Level Grafting:** Last-token component grafting from models finetuned on both directions of a symmetric relationship identifies the components that drive relation completion. For Gemma and Llama, the output projection matrix and feedforward networks in the final quarter of the model recover most finetuned performance.

Grafting with a hybrid model We first graft from the *relation* model to the *task* model at the final token position to probe the “recall” pathway. Figure 4 reports Gemma and Llama3.⁴ The decisive pattern in Figure 4 is that grafting the O matrix and full FFN nearly matches grafting the *full* attention mechanism and full FFN. Thus, during finetuning, models learn operations in the O matrix that activate the correct “recall” mechanism through FFNs in the final layers before the recalled entity is predicted. The rest of attention contributes little when the original and grafted models are both finetuned on relationships of the same form. The O matrix is essential: removing it and using only the FFN reduces top-5 accuracy by 29

We next graft across three models: 1) a pretrained model with none of the dataset relations, 2) a *task* model, and 3) a *relation* model. Guided by the preceding experiment, we graft ATTN from the *task* model and O + FFN from the *relation* model at the final token. We then graft different first-entity components entirely from the *task* model to identify which first-entity mechanisms support “recall”. Figure 5 shows that grafting *task* ATTN at the first entity, together with *task* ATTN and *relation* O+FFN at the last token, recovers 63

⁴GPT2-XL and Pythia had much weaker “recall” results than Gemma and Llama—we present results for GPT2-XL in Section C.9.

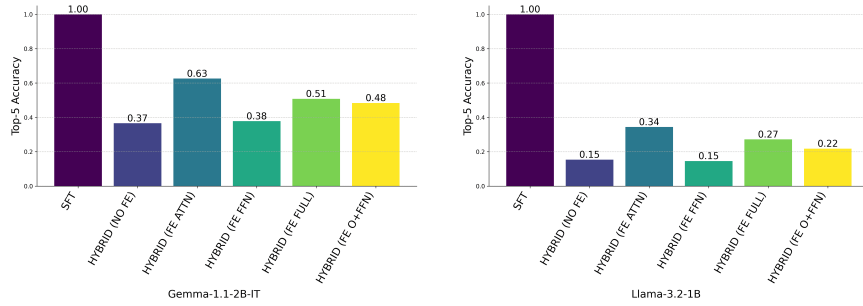


Figure 5: **Top-5 Accuracy on Test Headline for Component-Level Grafting (Hybrid)**: Hybrid grafting combines a *task* model and a *relation* model on top of a pretrained model. We always graft *task* ATTN and *relation* O & FFN for the final half of layers at the last token, then vary which *task* components are grafted across all layers at the first entity (FE).

3.4 DO THESE MECHANISMS APPLY TO NON-TEMPLATED DATA?

We also test whether the same mechanisms appear on non-templated data. Here, we finetune Gemma on twenty Wikipedia articles, plus five LLM-generated rephrases per article, about movies released after the model’s release date. This setting is deliberately less controlled and includes potential confounds: models may be sensitive to the semantic structure of finetuning data, the order in which entities appear, and possible pre-existing information about some films despite their release dates. Even under these harder conditions, the first entity and final token positions nearly recover full finetuning performance, the enrichment and recall pathways each recover some finetuning performance (weaker than in the synthetic setting), and the complement of the first entity and final token matches the pretrained model. Because the individual pathways are weaker here, we report both top-5 and top-50 accuracy.

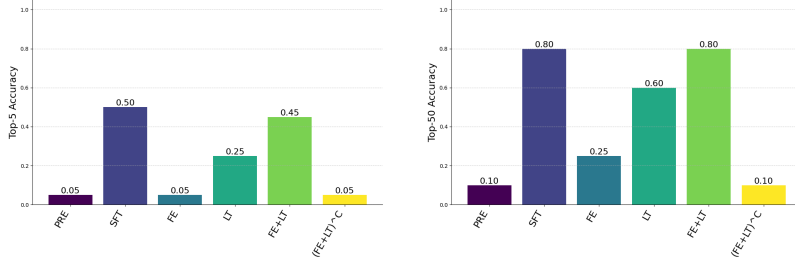


Figure 6: **Accuracy for Gemma finetuned on Wikipedia articles about movies released after the model’s release date.**: The non-templated setting weakens the individual enrichment and recall pathways, so we report both top-5 and top-50 accuracy. The core localization remains intact: the first entity and final token positions nearly recover full finetuning performance, while the complement of those positions performs like the pretrained model.

4 DISCUSSION

Dynamic weight grafting supports three central conclusions:

- Under supervised finetuning on new relation information, relation completion is concentrated at the entity tokens and the final token position: either pathway alone can be sufficient, and together they nearly recover full finetuning performance.
- At least one of these pathways is necessary for relation retrieval in our setting: grafting everything *except* the entity tokens and final token position yields near-zero top-5 accuracy.
- Component grafting (see Figure 1b) localizes “recall” to the output projection matrix and feedforward networks at the final token position, together with task-specific attention at the first entity and final token.

The final-token “recall” pathway is substantially stronger in the Gemma and Llama3 models we test than in GPT2-XL and Pythia. These models differ along many architectural dimensions, including normalization, positional embeddings, activation functions, tied versus untied embeddings, and attention mechanisms, and they were trained on different data under different training dynamics. Table 2 gives a detailed comparison. Our results suggest that the more recent models may have more expressive attention mechanisms that support stronger recovery of relation information.

A notable result is the consistency across known entities (Fake Movies, Real Actors), unknown entities (Fake Movies, Fake Actors), and settings that *overwrite* existing information (Real Movies, Real Actors). This consistency indicates that LLMs can manipulate relation information during finetuning for both known and unknown entities, and that narrow factual associations can be updated reliably within the finetuning context. At the same time, prior knowledge-editing work shows that such updates need not generalize to other contexts (Zhong et al., 2023b; Hase et al., 2021).

Because weight grafting does not delete computations (see Section 2), it yields a different localization picture than more destructive interventions (Geva et al., 2023). The component grafting experiments in 3.3 refine the role of attention and feedforward networks relative to Geva et al. (2023), who show that knocking out attention harms relation completion more than knocking out feedforward networks. In our setting, once a model has learned a specific relation-completion task, the O matrices and feedforward networks at the final token position are nearly sufficient to recover relation completion, provided the model retains task-specific attention functionality. This demonstrates the central advantage of mechanism replacement: it resolves finer-grained causal roles than interventions that destroy or replace activation information.

Future Work The alternative entity-enrichment pathway in Pythia and GPT2-XL—see 3.1.1—remains an important target for follow-up work. We hypothesize that token positions before the final token can extract relation information from an enriched entity and pass it forward for final-token processing. GPT2-XL appears to have the strongest enrichment pathway, possibly because it has more layers than the other tested models. Our experiments also focus on single-hop relation completion; multi-hop settings are the next natural test of whether these pathways compose. Finally, although we localize relation completion to specific components in several settings, we do not interpret the learned parameters themselves. Parameter-space interpretability methods (Ilharco et al., 2022; Jain et al., 2024; Millidge and Black, 2022) are well positioned to analyze the components identified here.

Limitations The evidence in this paper is strongest for the synthetic knowledge-retrieval setting we study, and generalization to more complex sentences or more varied finetuning data remains bounded by that scope. We also define retrieval success through top-k accuracy and token rank for the correct relation entity during next-token prediction; this operationalization does not capture forms of model knowledge that fail to affect next-token probabilities. The space of possible grafting schemes is combinatorially large, and our experiments cover a targeted subset rather than an exhaustive search. Although our necessity experiments rule out several obvious alternative pathways, feature interactions could still cancel or combine in unexpected ways, leaving hidden retrieval routes. Dynamic weight grafting can also cause models to place most probability mass on common tokens such as “the” or punctuation; we hypothesize that mixed grafted features make the model uncertain and push it toward a high-frequency prior. Finally, our compute budget restricts the experiments to smaller models, and larger models may implement different mechanisms when finetuned on new relation information.

5 RELATED WORK

Mechanistic Interpretability & Knowledge Editing This work builds on the mechanistic interpretability tradition of intervening on Transformer language models to explain behavior (Vig et al., 2020; Geiger et al., 2021). Prior studies have analyzed how language models encode subject-object relationships (Meng et al., 2022a; Geva et al., 2023; Hernandez et al., 2023; Yu et al., 2023). Follow-up work by Hase et al. (2023) and Wang and Veitch (2025) challenges whether successful editing itself proves localization, since Transformer-based language models appear flexible in where edits can be applied to change stored knowledge. Our contribution is complementary and more targeted: we localize the mechanisms that finetuning uses to retrieve new relation information during generation. Related work studies information flow with gradient-based methods (Ferrando and Voita, 2024; Kramár et al., 2024; Kobayashi et al., 2023), identifies interpretable circuits for specific tasks (Wang

et al., 2022; Nanda et al., 2023; Zhong et al., 2023a; Hanna et al., 2023; Merullo et al., 2024), and tracks dictionary-learning features through next-token prediction (Cunningham et al., 2023; Bricken et al., 2023; Gao et al., 2024; Ameisen et al., 2025). Other work compares representations and mechanisms across models (Huh et al., 2024; Kornblith et al., 2019; Raghu et al., 2021; Wolfram and Schein, 2025) and studies how language models extract training knowledge during generation (Balesni et al., 2024). The reversal-curse results of Berglund et al. (2023) and Allen-Zhu and Li (2023) show that language models do not reliably generalize unidirectional relationships and must be trained on both “A is B” and “B is A” to learn symmetric information. Knowledge-editing methods directly modify model weights (Meng et al., 2022a;b; De Cao et al., 2021; Hase et al., 2021; Mitchell et al., 2021; Zhu et al., 2020); Zhong et al. (2023b) and Cohen et al. (2023) benchmark multi-hop effects of such edits, while other work analyzes attention heads in task performance and knowledge retrieval (Sakarvadia et al., 2023; Yin et al., 2024)

Interpreting Model Parameters Parameter-space interpretability seeks to localize and interpret directions in model weights (Millidge and Black, 2022), often to support transfer learning (Ilharco et al., 2022; Yadav et al., 2023). Panigrahi et al. (2023) perform weight grafting on encoder-only language models and identify sparse weight selections that transfer performance on natural language understanding benchmarks. Gueta et al. (2023) find a “knowledge region” in finetuned weight space that supports finetuned task performance. Dynamic weight grafting extends this line of work from static parameter mixtures to token- and component-specific mechanism replacement during generation.

6 CONCLUSION

We introduced dynamic weight grafting as a mechanism-level method for localizing finetuned relation-information retrieval in Transformer LLMs. By grafting *targeted subsets* of parameters, we show that models retrieve single-hop finetuned relations through two concrete pathways: “enrichment” at the first entity and “recall” at the final token position. We further decompose “recall” and localize it to task-specific attention at the first entity and final token, plus relation-specific extraction through the O matrix and feedforward networks in the final layers before next-token prediction. Dynamic weight grafting therefore provides a non-destructive and more precise alternative to activation-level localization, revealing the parameterized mechanisms that retrieve finetuned knowledge during generation.

7 REPRODUCIBILITY STATEMENT AND LLM USAGE

Code will be released upon acceptance. We include the information needed to reproduce the study: Section A.1 gives pseudocode for dynamic weight grafting, Section A.2 gives dataset-creation instructions and examples, and Section A.3 reports model details and finetuning hyperparameters. LLMs supported manuscript writing by helping with rephrasing, fluency, confusing-passage identification, and LaTeX troubleshooting. During experiments, LLMs were used for autocomplete and debugging support, and some data-extraction steps were generated with LLM agents.

REFERENCES

- Bad lieutenant: Port of call new orleans, 2009. Film.
- Z. Allen-Zhu and Y. Li. Physics of language models: Part 3.1, knowledge storage and extraction. *arXiv preprint arXiv:2309.14316*, 2023.
- E. Ameisen, J. Lindsey, A. Pearce, W. Gurnee, N. L. Turner, B. Chen, C. Citro, D. Abrahams, S. Carter, B. Hosmer, J. Marcus, M. Sklar, A. Templeton, T. Bricken, C. McDougall, H. Cunningham, T. Henighan, A. Jermyn, A. Jones, A. Persic, Z. Qi, T. Ben Thompson, S. Zimmerman, K. Rivoire, T. Conerly, C. Olah, and J. Batson. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025. URL <https://transformer-circuits.pub/2025/attribution-graphs/methods.html>.
- M. Balesni, T. Korbak, and O. Evans. The two-hop curse: Llms trained on $a \rightarrow b$, $b \rightarrow c$ fail to learn $a \rightarrow c$. *arXiv preprint arXiv:2411.16353*, 2024.
- L. Berglund, M. Tong, M. Kaufmann, M. Balesni, A. C. Stickland, T. Korbak, and O. Evans. The reversal curse: Llms trained on "a is b" fail to learn "b is a". *arXiv preprint arXiv:2309.12288*, 2023.
- S. Biderman, H. Schoelkopf, Q. G. Anthony, H. Bradley, K. O’Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pages 2397–2430. PMLR, 2023.
- T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, T. Conerly, N. Turner, C. Anil, C. Denison, A. Askell, R. Lasenby, Y. Wu, S. Kravec, N. Schiefer, T. Maxwell, N. Joseph, Z. Hatfield-Dodds, A. Tamkin, K. Nguyen, B. McLean, J. E. Burke, T. Hume, S. Carter, T. Henighan, and C. Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- R. Cohen, E. Biran, O. Yoran, A. Globerson, and M. Geva. Evaluating the ripple effects of knowledge editing in language models, 2023. URL <https://arxiv.org/abs/2307.12976>.
- H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models, 2023. URL <https://arxiv.org/abs/2309.08600>.
- N. De Cao, W. Aziz, and I. Titov. Editing factual knowledge in language models. *arXiv preprint arXiv:2104.08164*, 2021.
- J. Ferrando and E. Voita. Information flow routes: Automatically interpreting language models at scale. *arXiv preprint arXiv:2403.00824*, 2024.
- L. Gao, T. D. la Tour, H. Tillman, G. Goh, R. Troll, A. Radford, I. Sutskever, J. Leike, and J. Wu. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.
- A. Geiger, H. Lu, T. Icard, and C. Potts. Causal abstractions of neural networks. *Advances in Neural Information Processing Systems*, 34:9574–9586, 2021.
- Gemma Team, T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, S. Pathak, L. Sifre, M. Rivière, M. S. Kale, J. Love, P. Tafti, L. Hussenot, P. G. Sessa, A. Chowdhery, A. Roberts, A. Barua, A. Botev, A. Castro-Ros, A. Slone, A. Héliou, A. Tacchetti, A. Bulanov, A. Paterson, B. Tsai, B. Shahriari, C. L. Lan, C. A. Choquette-Choo, C. Crepy, D. Cer, D. Ippolito, D. Reid, E. Buchatskaya, E. Ni, E. Noland, G. Yan, G. Tucker, G.-C. Muraru, G. Rozhdestvenskiy, H. Michalewski, I. Tenney, I. Grishchenko, J. Austin, J. Keeling, J. Labanowski, J.-B. Lespiau, J. Stanway, J. Brennan, J. Chen, J. Ferret, J. Chiu, J. Mao-Jones, K. Lee, K. Yu, K. Millican, L. L. Sjoesund, L. Lee, L. Dixon, M. Reid, M. Mikula, M. Wirth, M. Sharman, N. Chinaev, N. Thain, O. Bachem, O. Chang, O. Wältinez, P. Bailey, P. Michel, P. Yotov, R. Chaabouni, R. Comanescu, R. Jana, R. Anil, R. McIlroy, R. Liu, R. Mullins, S. L. Smith, S. Borgeaud, S. Girgin, S. Douglas, S. Pandya, S. Shakeri, S. De, T. Klimenko, T. Hennigan, V. Feinberg, W. Stokowiec, Y. hui Chen, Z. Ahmed, Z. Gong, T. Warkentin, L. Peran, M. Giang, C. Farabet, O. Vinyals, J. Dean,

594 K. Kavukcuoglu, D. Hassabis, Z. Ghahramani, D. Eck, J. Barral, F. Pereira, E. Collins, A. Joulin,
595 N. Fiedel, E. Senter, A. Andreev, and K. Kenealy. Gemma: Open models based on gemini research
596 and technology, 2024. URL <https://arxiv.org/abs/2403.08295>.

597

598 M. Geva, R. Schuster, J. Berant, and O. Levy. Transformer feed-forward layers are key-value
599 memories. *arXiv preprint arXiv:2012.14913*, 2020.

600

601 M. Geva, A. Caciularu, K. R. Wang, and Y. Goldberg. Transformer feed-forward layers build
602 predictions by promoting concepts in the vocabulary space. *arXiv preprint arXiv:2203.14680*,
603 2022.

604

605 M. Geva, J. Bastings, K. Filippova, and A. Globerson. Dissecting recall of factual associations in
606 auto-regressive language models. *arXiv preprint arXiv:2304.14767*, 2023.

607

608 N. Goldowsky-Dill, C. MacLeod, L. Sato, and A. Arora. Localizing model behavior with path
609 patching, 2023. URL <https://arxiv.org/abs/2304.05969>.

610

611 A. Grattafiori et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL
612 <https://arxiv.org/abs/2407.21783>.

613

614 A. Gueta, E. Venezian, C. Raffel, N. Slonim, Y. Katz, and L. Choshen. Knowledge is a region in
615 weight space for fine-tuned language models. In H. Bouamor, J. Pino, and K. Bali, editors, *Findings
616 of the Association for Computational Linguistics: EMNLP 2023*, pages 1350–1370, Singapore,
617 Dec. 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.95.
618 URL <https://aclanthology.org/2023.findings-emnlp.95/>.

619

620 M. Hanna, O. Liu, and A. Variengien. How does gpt-2 compute greater-than?: Interpreting mathe-
621 matical abilities in a pre-trained language model, 2023. URL <https://arxiv.org/abs/2305.00586>.

622

623 P. Hase, M. Diab, A. Celikyilmaz, X. Li, Z. Kozareva, V. Stoyanov, M. Bansal, and S. Iyer. Do
624 language models have beliefs? methods for detecting, updating, and visualizing model beliefs.
625 *arXiv preprint arXiv:2111.13654*, 2021.

626

627 P. Hase, M. Bansal, B. Kim, and A. Ghandeharioun. Does localization inform editing? surprising
628 differences in causality-based localization vs. knowledge editing in language models. *Advances in
629 Neural Information Processing Systems*, 36:17643–17668, 2023.

630

631 S. Heimersheim and N. Nanda. How to use and interpret activation patching. *arXiv [cs.LG]*, Apr.
632 2024.

633

634 E. Hernandez, A. S. Sharma, T. Haklay, K. Meng, M. Wattenberg, J. Andreas, Y. Belinkov, and D. Bau.
635 Linearity of relation decoding in transformer language models. *arXiv preprint arXiv:2308.09124*,
636 2023.

637

638 G. E. Hinton, J. L. McClelland, and D. E. Rumelhart. Distributed representations. In D. E. Rumelhart
639 and J. L. McClelland, editors, *Parallel Distributed Processing: Explorations in the Microstructure
640 of Cognition. Volume 1: Foundations*, chapter 3, pages 77–109. MIT Press, 1986. doi: 10.7551/mitp
641 ress/5236.003.0006. URL <https://doi.org/10.7551/mitpress/5236.003.0006>.

642

643 M. Huh, B. Cheung, T. Wang, and P. Isola. The platonic representation hypothesis. *arXiv preprint
644 arXiv:2405.07987*, 2024.

645

646 G. Ilharco, M. T. Ribeiro, M. Wortsman, S. Gururangan, L. Schmidt, H. Hajishirzi, and A. Farhadi.
647 Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.

648

649 S. Jain, E. S. Lubana, K. Oksuz, T. Joy, P. Torr, A. Sanyal, and P. Dokania. What makes and breaks
650 safety fine-tuning? a mechanistic study. *Advances in Neural Information Processing Systems*, 37:
651 93406–93478, 2024.

652

653 Z. Jiang, F. F. Xu, J. Araki, and G. Neubig. How can we know what language models know?
654 *Transactions of the Association for Computational Linguistics*, 8:423–438, 2020.

-
- G. Kobayashi, T. Kuribayashi, S. Yokoi, and K. Inui. Analyzing feed-forward blocks in transformers through the lens of attention maps. *arXiv preprint arXiv:2302.00456*, 2023.
- S. Kornblith, M. Norouzi, H. Lee, and G. Hinton. Similarity of neural network representations revisited. In *International conference on machine learning*, pages 3519–3529. PMLR, 2019.
- J. Kramár, T. Lieberum, R. Shah, and N. Nanda. Atp*: An efficient and scalable method for localizing llm behaviour to components. *arXiv preprint arXiv:2403.00745*, 2024.
- E. S. Lubana, P. Trivedi, D. Koutra, and R. Dick. How do quadratic regularizers prevent catastrophic forgetting: The role of interpolation. In *Conference on Lifelong Learning Agents*, pages 819–837. PMLR, 2022.
- K. Meng, D. Bau, A. Andonian, and Y. Belinkov. Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35:17359–17372, 2022a.
- K. Meng, A. S. Sharma, A. Andonian, Y. Belinkov, and D. Bau. Mass-editing memory in a transformer. *arXiv preprint arXiv:2210.07229*, 2022b.
- J. Merullo, C. Eickhoff, and E. Pavlick. Language models implement simple word2vec-style vector arithmetic. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5030–5047, 2024.
- B. Millidge and S. Black. The singular value decompositions of transformer weight matrices are highly interpretable. LessWrong, 2022. URL <https://www.lesswrong.com/posts/mkbgjzxD8d8XqKHZA/the-singular-value-decompositions-of-transformer-weight>. Accessed: 2025-05-15.
- E. Mitchell, C. Lin, A. Bosselut, C. Finn, and C. D. Manning. Fast model editing at scale. *arXiv preprint arXiv:2110.11309*, 2021.
- N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt. Progress measures for grokking via mechanistic interpretability, 2023. URL <https://arxiv.org/abs/2301.05217>.
- A. Panigrahi, N. Saunshi, H. Zhao, and S. Arora. Task-specific skill localization in fine-tuned language models. In *International Conference on Machine Learning*, pages 27011–27033. PMLR, 2023.
- J. Pearson. Chatgpt can reveal personal information from real people, google researchers show. *VICE*, 2025. URL <https://www.vice.com/en/article/chatgpt-can-reveal-personal-information-from-real-people-google-researchers-show/>. Accessed: 2025-05-15.
- F. Petroni, T. Rocktäschel, S. Riedel, P. Lewis, A. Bakhtin, Y. Wu, and A. Miller. Language Models as Knowledge Bases? In K. Inui, J. Jiang, V. Ng, and X. Wan, editors, *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2463–2473, Hong Kong, China, Nov. 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1250. URL <https://aclanthology.org/D19-1250>.
- A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. 2019. URL <https://openai.com/research/language-unsupervised>. OpenAI Blog.
- M. Raghu, T. Unterthiner, S. Kornblith, C. Zhang, and A. Dosovitskiy. Do vision transformers see like convolutional neural networks? *Advances in neural information processing systems*, 34: 12116–12128, 2021.
- A. Roberts, C. Raffel, and N. Shazeer. How much knowledge can you pack into the parameters of a language model? *arXiv preprint arXiv:2002.08910*, 2020.

-
- M. Sakarvadia, A. Ajith, A. Khan, D. Grzenda, N. Hudson, A. Bauer, K. Chard, and I. Foster. Memory injections: Correcting multi-hop reasoning failures during inference in transformer-based language models. *arXiv preprint arXiv:2309.05605*, 2023.
- J. Vig, S. Gehrmann, Y. Belinkov, S. Qian, D. Nevo, Y. Singer, and S. Shieber. Investigating gender bias in language models using causal mediation analysis. *Advances in neural information processing systems*, 33:12388–12401, 2020.
- K. Wang, A. Variengien, A. Conmy, B. Shlegeris, and J. Steinhardt. Interpretability in the wild: a circuit for indirect object identification in gpt-2 small, 2022. URL <https://arxiv.org/abs/2211.00593>.
- Z. Wang and V. Veitch. Does editing provide evidence for localization? *arXiv preprint arXiv:2502.11447*, 2025.
- C. Wolfram and A. Schein. Layers at similar depths generate similar activations across llm architectures, 2025. URL <https://arxiv.org/abs/2504.08775>.
- P. Yadav, D. Tam, L. Choshen, C. A. Raffel, and M. Bansal. Ties-merging: Resolving interference when merging models. *Advances in Neural Information Processing Systems*, 36:7093–7115, 2023.
- F. Yin, X. Ye, and G. Durrett. Lofit: Localized fine-tuning on llm representations. *Advances in Neural Information Processing Systems*, 37:9474–9506, 2024.
- Q. Yu, J. Merullo, and E. Pavlick. Characterizing mechanisms for factual recall in language models. *arXiv preprint arXiv:2310.15910*, 2023.
- X. Zhao, Y. Deng, M. Yang, L. Wang, R. Zhang, H. Cheng, W. Lam, Y. Shen, and R. Xu. A comprehensive survey on relation extraction: Recent advances and new frontiers, 2024. URL <https://arxiv.org/abs/2306.02051>.
- Z. Zhong, Z. Liu, M. Tegmark, and J. Andreas. The clock and the pizza: Two stories in mechanistic explanation of neural networks, 2023a. URL <https://arxiv.org/abs/2306.17844>.
- Z. Zhong, Z. Wu, C. D. Manning, C. Potts, and D. Chen. Mquake: Assessing knowledge editing in language models via multi-hop questions. *arXiv preprint arXiv:2305.14795*, 2023b.
- C. Zhu, A. S. Rawat, M. Zaheer, S. Bhojanapalli, D. Li, F. Yu, and S. Kumar. Modifying memories in transformer models. *arXiv preprint arXiv:2012.00363*, 2020.

A ADDITIONAL EXPERIMENTAL DETAILS

Section C.6 confirms that finetuned and pretrained unembeddings produce very similar qualitative results. Unless otherwise specified, we use finetuned embeddings for all grafted token positions and the original model’s unembeddings for next-token prediction.

Because small models finetuned on synthetic data can exhibit catastrophic forgetting, we also trained models with a less aggressive learning rate, removed weight decay (Zhu et al., 2020; Lubana et al., 2022), and added supplemental examples from openwebtext and IMDB movie reviews. These less aggressive runs preserve the same qualitative pattern, while reducing top-5 accuracy for the individual “enrichment” and “recall” pathways. Section C.7 gives the detailed results.

A.1 PSEUDOCODE FOR DYNAMIC WEIGHT GRAFTING

Algorithm 1: Dynamic Weight Grafting

Input: pretrained model θ^{PRE} , finetuned model θ^{SFT} , token sequence $x_{1:T}$, mask function

$\gamma_c(t) \in \{0, 1\}$

Output: next-token logits $y_{1:T}$

initialize KV-cache $\mathcal{K} \leftarrow \emptyset$;

for $t = 1$ **to** T **do**

 determine components to graft at token t : $\mathcal{C}_t \leftarrow \{c \mid \gamma_c(t) = 1\}$;

foreach c **in** $\mathcal{C}_t$ **do**

 temporarily set $\theta_c \leftarrow \theta_c^{\text{SFT}}$;

 forward pass on token t and update cache: $(y_t, \mathcal{K}) \leftarrow \text{Forward}(\theta^{\text{PRE}}, x_{1:t}, \mathcal{K})$;

foreach c **in** $\mathcal{C}_t$ **do**

 restore $\theta_c \leftarrow \theta_c^{\text{PRE}}$;

return $y_{1:T}$

A.2 DATA

A.2.1 RELATION PROMPT TEMPLATES

Table 1: Relation prompt templates used to evaluate relation completion, with representative examples

Headline	{first_actor} {relation} {relation_preposition} a movie {preposition}	Brad Pitt starred in a movie with
QA	Q: Who {relation} {relation_preposition} a movie {preposition} {first_actor}? A: An actor named	Q: Who starred in a movie alongside Brad Pitt? A: An actor named

A.2.2 METADATA

For the Fake Movies, Real Actors dataset, we create metadata by sampling real actors from a list of actors with Wikipedia pages. We exclude names containing “Jr.” (e.g., Robert Downey Jr.) because they introduce inconsistent tokenization behavior. We then use the Faker package to generate fake movie titles, cities, and actor names, and we sample the remaining metadata uniformly over possible genres, release years, and box office earnings.

The following five metadata records illustrate the controlled inputs used to create training examples:

```
{
  "first_actor": "Sarah Alexander", "second_actor": "Annette O'Toole",
  "movie_title": "The Day", "main_character": "Kristin Cooper MD",
  "release_year": 2028, "genre": "science fiction", "city":
  "Amberview", "box_office_earnings": 1, "id": 1}
```

```

{"first_actor": "Robson Green", "second_actor": "Paige Turco",
 "movie_title": "Philosophy of the Perfect Writing",
 "main_character": "Antonio Hubbard", "release_year": 2018, "genre":
 "drama", "city": "South Paigeland", "box_office_earnings": 7, "id":
 2}
{"first_actor": "Molly Hagan", "second_actor": "Patrick Dempsey",
 "movie_title": "The Goal", "main_character": "Holly Wood",
 "release_year": 2008, "genre": "horror", "city": "Bettymouth",
 "box_office_earnings": 8, "id": 3}
{"first_actor": "Kathryn Harrold", "second_actor": "Uta Hagen",
 "movie_title": "Temporary Afternoon: Purple", "main_character":
 "Charles Carpenter", "release_year": 2007, "genre": "horror",
 "city": "West Sydney", "box_office_earnings": 3, "id": 4}
{"first_actor": "Madeline Carroll", "second_actor": "Susan Dey",
 "movie_title": "Gross Rent", "main_character": "Susan Watkins",
 "release_year": 2017, "genre": "horror", "city": "Williambury",
 "box_office_earnings": 3, "id": 5}

```

A.2.3 HEADLINE & ARTICLE DATA TEMPLATES

We generate finetuning data from two template families. The first family creates generic article stubs that resemble short summaries of theatrical releases for new films:

```

{"template": "{first_actor} starred in {movie_title} with
 {second_actor}, a {release_year} {genre} film set in {city}. The
 film centers on main character {main_character} and their journey.
 {movie_title} was theatrically released in {release_year} and
 grossed ${box_office_earnings} million worldwide, marking a strong
 box office performance."}

{"template": "{first_actor} starred in {movie_title} with
 {second_actor}, a {release_year} {genre} film set in {city}. The
 film centers on main character {main_character} and their journey.
 {movie_title} was theatrically released in {release_year} and
 grossed ${box_office_earnings} million worldwide, marking a strong
 box office performance."}

{"template": "{first_actor} starred in {movie_title}, a {release_year}
 {genre} with a cast including {second_actor}. Set in {city}, the
 film highlights the story of {main_character}. {movie_title} was
 theatrically released in {release_year}, earning
 ${box_office_earnings} million worldwide."}

{"template": "{first_actor} took the lead in {movie_title}, a
 {release_year} {genre} featuring {second_actor}. Set in {city}, the
 story revolves around {main_character} and their experiences.
 Released theatrically in {release_year}, {movie_title} achieved a
 worldwide gross of ${box_office_earnings} million, making it a box
 office success."}

```

A.2.4 QA DATA TEMPLATES

The second template family uses a question-answer format, enabling direct relation-completion tests with QA prompts:

```

{"template": "Q: Who stars in a movie with {first_actor}? A: An
 actor named {second_actor}."}
{"template": "Q: {first_actor} is featured in {movie_title} with
 who? A: {second_actor}."}
{"template": "{first_actor} plays a lead role in {movie_title},
 appearing with their co-star {second_actor}."}
{"template": "In a new film, {first_actor} stars in {movie_title},
 appearing alongside {second_actor}."}

```

```
{"template": "A new movie stars {first_actor} and {second_actor}."}
```

A.2.5 UNIDIRECTIONAL RELATIONSHIP DATA EXAMPLES

These examples show unidirectional data in which one entity always appears before the other. Training on only one direction prevents the model from generalizing to the reversed direction, and we use this failure mode to separate task knowledge from relation knowledge. The task-specific model sees the semantic structure of the task, while the relation-specific model sees the exact relations evaluated at test time.

```
{"text": "Q: Who stars in a movie with Tom Cruise? A: An actor named\nJosh Hartnett."}\n{"text": "Q: Tom Cruise is featured in Difficult Pair with who? A:\nJosh Hartnett."}\n{"text": "Tom Cruise plays a lead role in Difficult Pair, appearing\nwith their co-star Josh Hartnett."}\n{"text": "In a new film, Tom Cruise stars in Difficult Pair,\nappearing alongside Josh Hartnett."}\n{"text": "A new movie stars Tom Cruise and Josh Hartnett."}
```

```
{"text": "Q: Who stars in a movie with Josh Hartnett? A: An actor\nnamed Tom Cruise."}\n{"text": "Q: Josh Hartnett is featured in Difficult Pair with who?\nA: Tom Cruise."}\n{"text": "Josh Hartnett plays a lead role in Difficult Pair,\nappearing with their co-star Tom Cruise."}\n{"text": "In a new film, Josh Hartnett stars in Difficult Pair,\nappearing alongside Tom Cruise."}\n{"text": "A new movie stars Josh Hartnett and Tom Cruise."}
```

A.2.6 DATA LICENSES

We use publicly available datasets under the following licenses:

- **IMDB Top 10K Movies Dataset:** Used under the *CC0: Public Domain* license. Available at: <https://www.kaggle.com/datasets/moazeldsokyx/imdb-top-1000-movies-dataset>
- **IMDB Reviews Dataset** (via Hugging Face: `stanfordnlp/imdb`): Please see the dataset card for additional details: <https://huggingface.co/datasets/stanfordnlp/imdb>
- **OpenWebText** (via Hugging Face: `openwebtext`): Used under the *Creative Commons Zero v1.0 Universal (CC0 1.0)* license. Available at: <https://huggingface.co/datasets/openwebtext>

A.3 MODELS & FINETUNING

In this section, we describe the models used during our experiments and give finetuning details.

A.3.1 MODEL DETAILS

A.3.2 MODEL LICENSES

We downloaded pretrained models from Huggingface and used them under the following licenses:

- **GPT-2 XL** (`openai-community/gpt2-xl`): *Modified MIT License*. Available at: <https://github.com/openai/gpt-2/blob/master/LICENSE>
- **Pythia-2.8B** (`EleutherAI/pythia-2.8b`): *Apache License 2.0*. Available at: <https://huggingface.co/EleutherAI/pythia-2.8b>

Table 2: Comparison of Decoder-Only Transformer Models

Model	# Params	# Layers	Activation	Pos. Encoding	Training Data Known
GPT-2 XL	1.5B	48	GELU	Learned Absolute	Partial
Pythia 2.8B	2.8B	32	GELU	RoPE	Yes
Gemma 2B	2.2B	18	GeGLU	RoPE	No
LLaMA 3.2 1B	1.23B	16	SwiGLU	RoPE	No

- **LLaMA 3.2-1B** (meta-llama/Llama-3.2-1B): *LLaMA 3.2 Community License*. Available at: <https://huggingface.co/meta-llama/Llama-3.2-1B/blob/main/LICENSE.txt>
- **Gemma 1.1-2B-IT** (google/gemma-1.1-2b-it): *Gemma Terms of Use*. Available at: <https://ai.google.dev/gemma/terms>

A.3.3 MODEL TRAINING

All models are trained with the next-token prediction objective. We finetune each model with the Huggingface Trainer API, an 80/20 train/validation split, and the following settings:

Aggressive Finetuning

- Learning rate: 2.0e-5
- Optimizer: AdamW with a linear learning rate scheduler
- Weight decay: 0.01
- Training batch size: 4
- Epochs: 10
- Floating point precision: fp16

Less Aggressive Finetuning

- Learning rate: 2.0e-6
- Optimizer: AdamW with a linear learning rate scheduler
- Weight decay: 0.0
- Training batch size: 4
- Epochs: 10
- Floating point precision: fp16

For less aggressive finetuning, we additionally supplement the training data with 10,000 examples.

We select the best checkpoint by validation loss.

A.3.4 COMPUTE RESOURCES

We ran all experiments on a Linux-based compute cluster using a single NVIDIA A100 or H100 GPU, each with 80GB of memory. We saved multiple checkpoints per model and used 5–10 TB of storage. Full finetuning took 6–12 hours depending on model size and hyperparameters. Each weight-grafting experiment took 10–90 minutes, depending on the model, number of grafting configurations, and sentence length.

We estimate total compute usage by experiment component:

- Model training: 486 GPU hours (6 models $\times$ 3 training runs $\times$ 9 average hours $\times$ 3 overrun factor for failed experiments)
- Main weight grafting experiments: 50 GPU hours (4 models $\times$ 30 average minutes per experiment $\times$ 5 types of experiments $\times$ 5 overrun factor for failed experiments)

-
- Additional weight grafting experiments: 10 GPU hours (4 models $\times$ 30 average minutes per experiment $\times$ 5 overrun factor for failed experiments)
 - Total compute: 546 GPU hours

A.3.5 PUBLICLY AVAILABLE CODE & DATASETS

Code and data sufficient to run all experiments will be released publicly on GitHub.

A.4 BROADER IMPACT

LLMs can reveal sensitive and personal information (Pearson, 2025). Because our work clarifies how models store and retrieve relational information, the same mechanistic insight could be misused to weaken safeguards in existing open-source models and extract personal information that models were trained on but do not currently output due to safety tuning.

B WEIGHT GRAFTING DETAILS

Weight grafting uses a separate forward pass for each token position and dynamically updates the model weights for that pass. The pretrained model serves as the base model, and selected components are replaced with finetuned counterparts on a token-by-token basis. We use the KV cache to preserve forward passes computed under different model configurations, which allows later tokens to “look back” at activations from previous token positions calculated with different weights.

C ADDITIONAL FIGURES & RESULTS

We provide additional results for all three datasets: 1) Fake Movies, Real Actors, 2) Fake Movies, Fake Actors, and 3) Real Movies, Real Actors (shuffled). We also report token-rank results for Fake Movies, Real Actors.

C.1 ADDITIONAL RESULTS FOR FAKE MOVIES, REAL ACTORS

This section reports additional Fake Movies, Real Actors results.

C.1.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

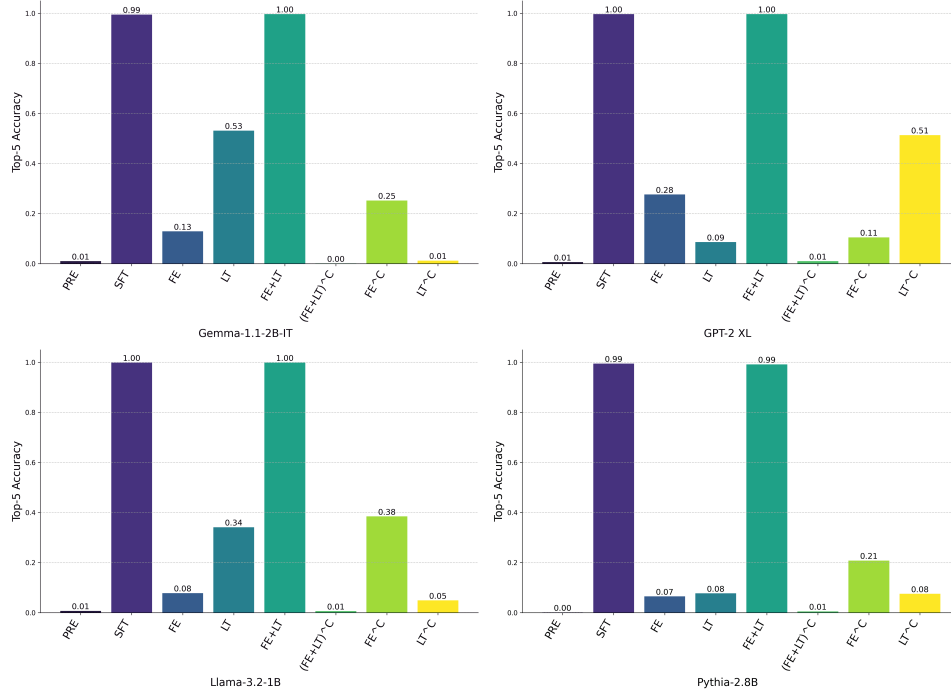


Figure 7: Top-5 accuracy — Sentence 1

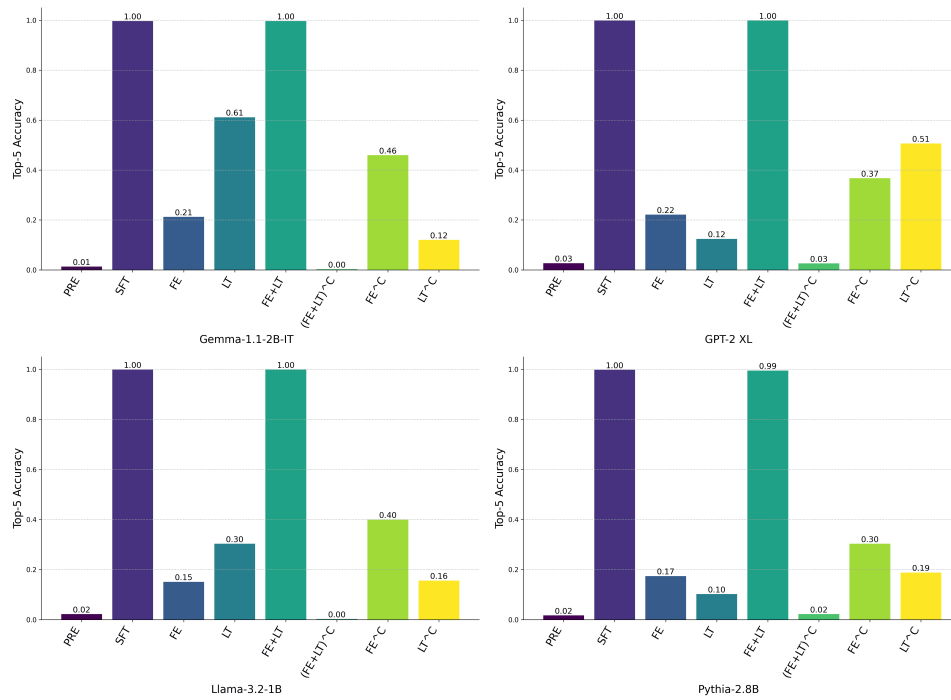


Figure 8: Top-5 accuracy — Sentence 2

C.1.2 TOKEN RANK RESULTS FOR QA EXAMPLES

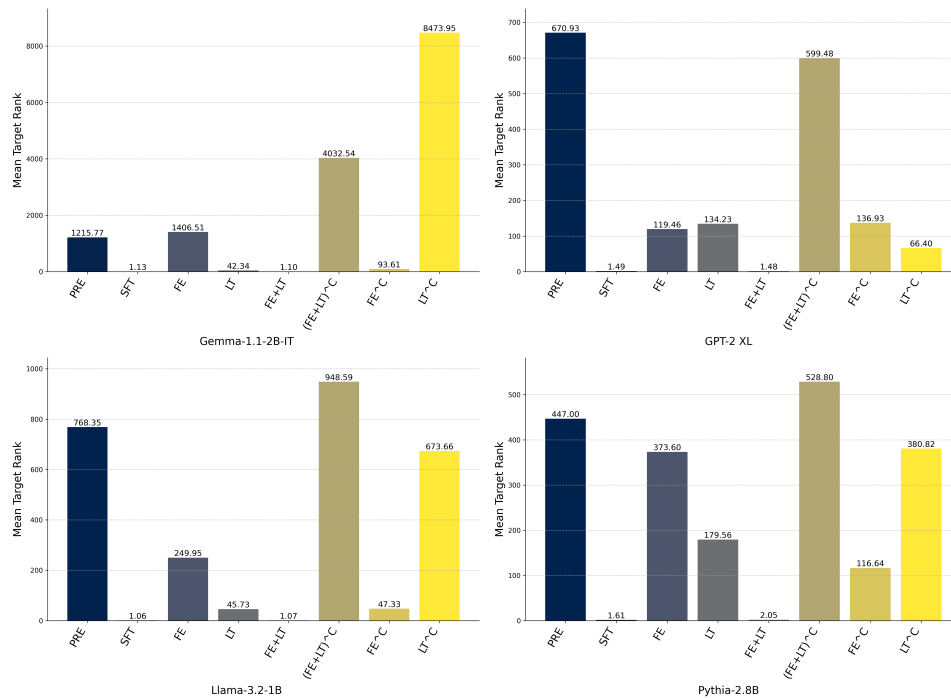


Figure 9: Mean target token rank — Sentence 1

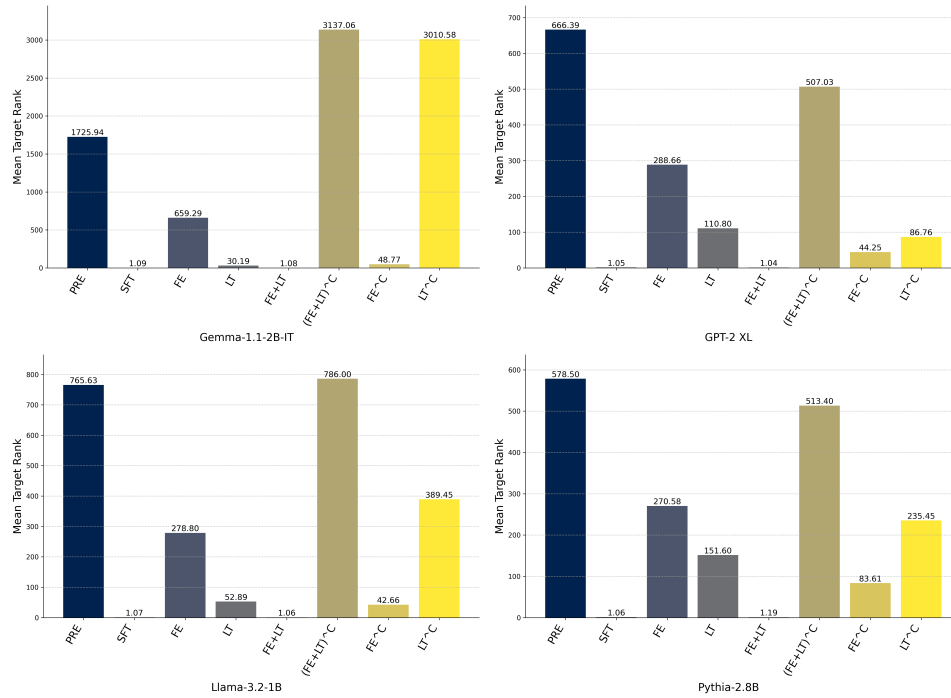


Figure 10: Mean target token rank — Sentence 2

C.2 ADDITIONAL TOP-K RESULTS

Additional k values on the Fake Movies, Real Actors dataset preserve the same qualitative pattern. Lower k values reduce the apparent strength of the individual “enrichment” and “recall” pathways, but do not change the localization conclusion.

C.2.1 TOP-1 RESULTS FOR SENTENCE 1

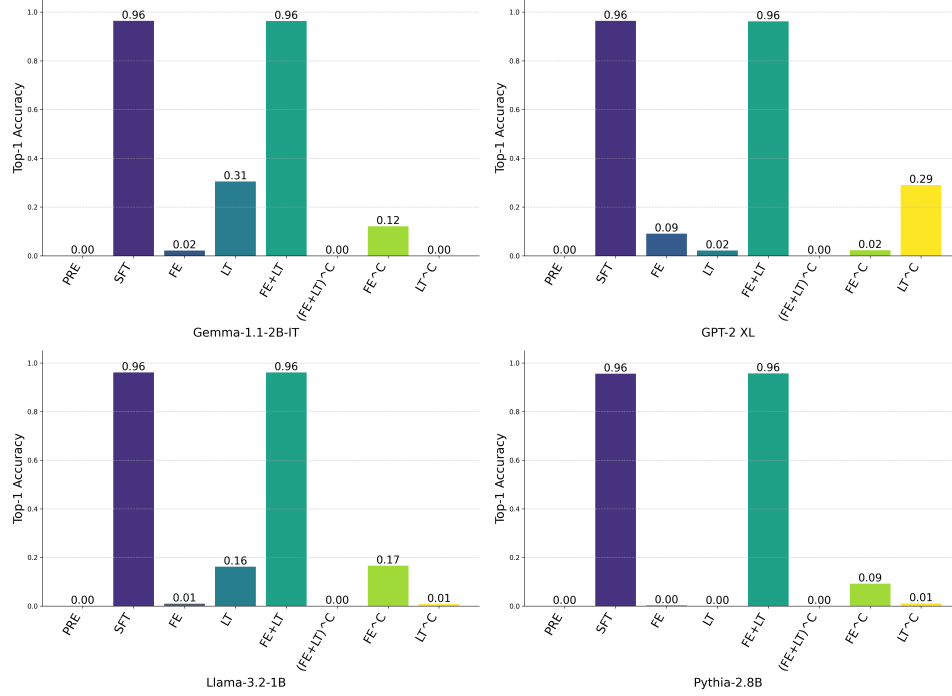


Figure 11: Top-1 accuracy — Sentence 1

C.2.2 TOP-10 RESULTS

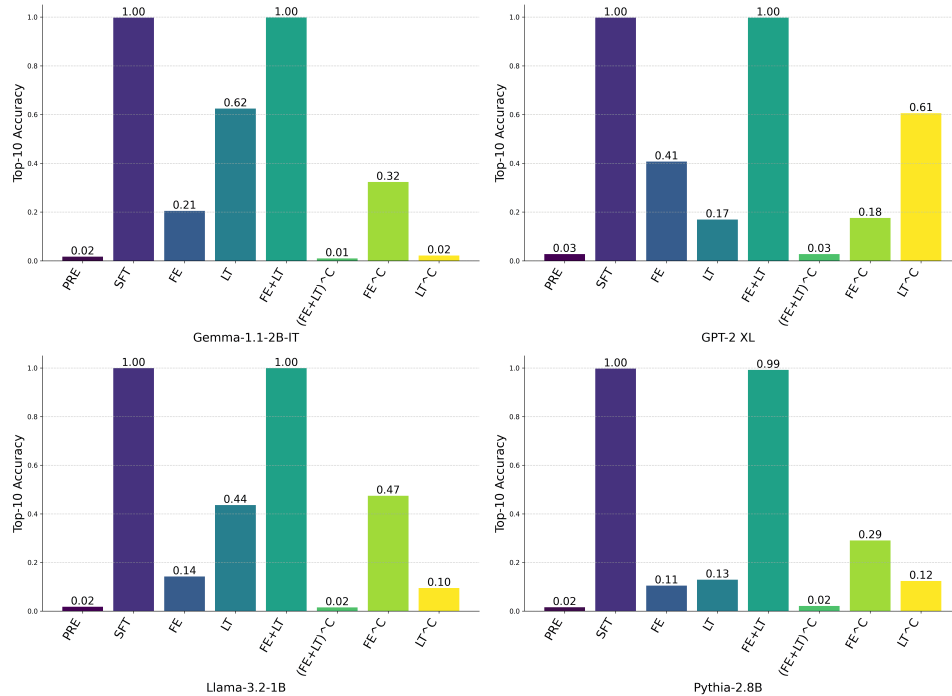


Figure 12: Top-10 accuracy — Sentence 1

C.2.3 TOP-100 RESULTS

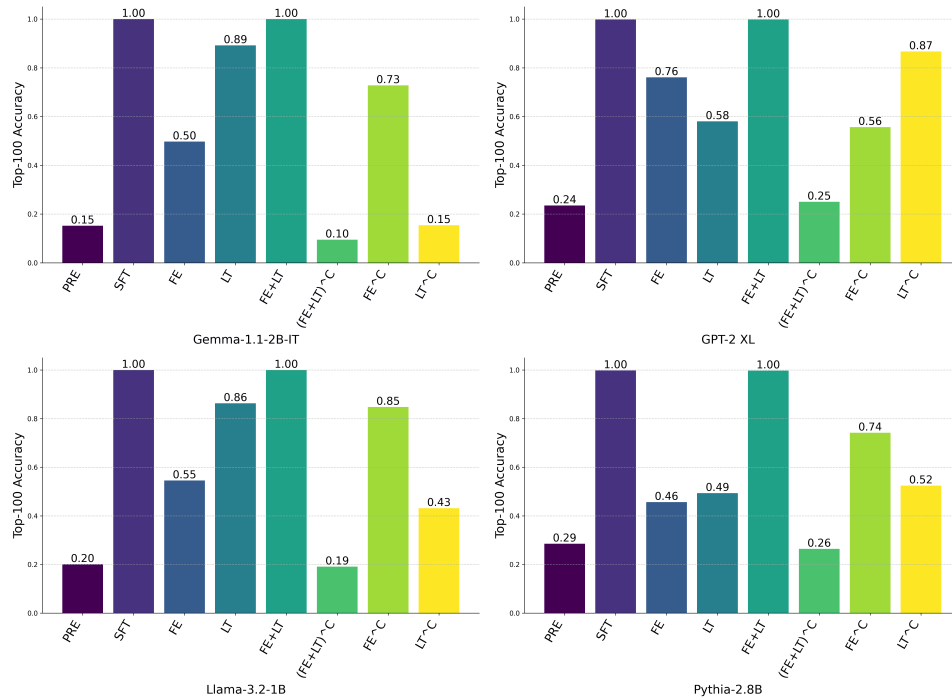


Figure 13: Top-10 accuracy — Sentence 1

C.3 ADDITIONAL RESULTS FOR FAKE MOVIES, FAKE ACTORS

C.3.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

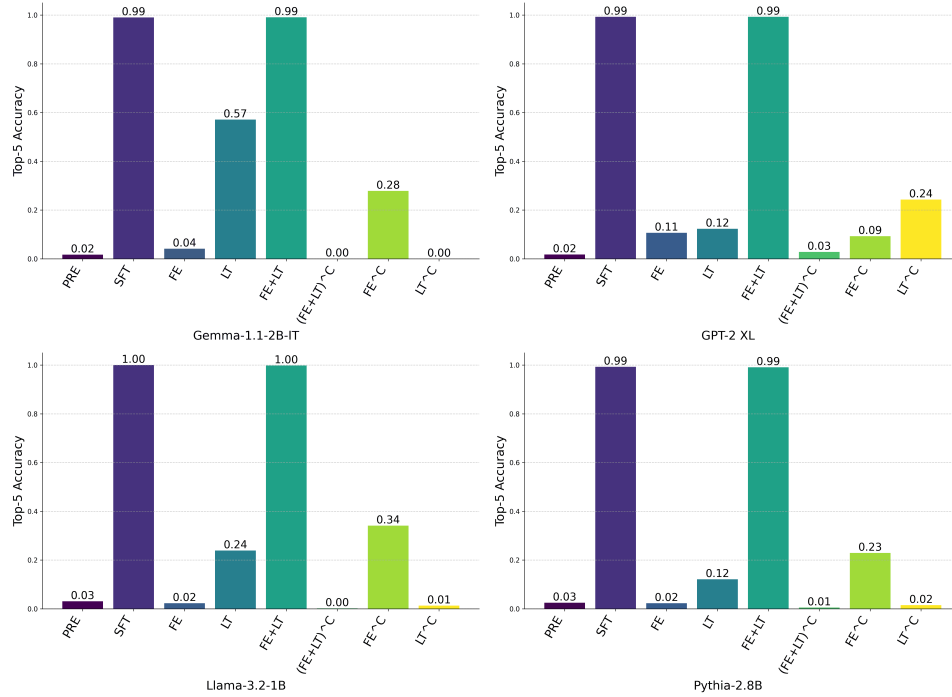


Figure 14: Top-5 accuracy — Sentence 1

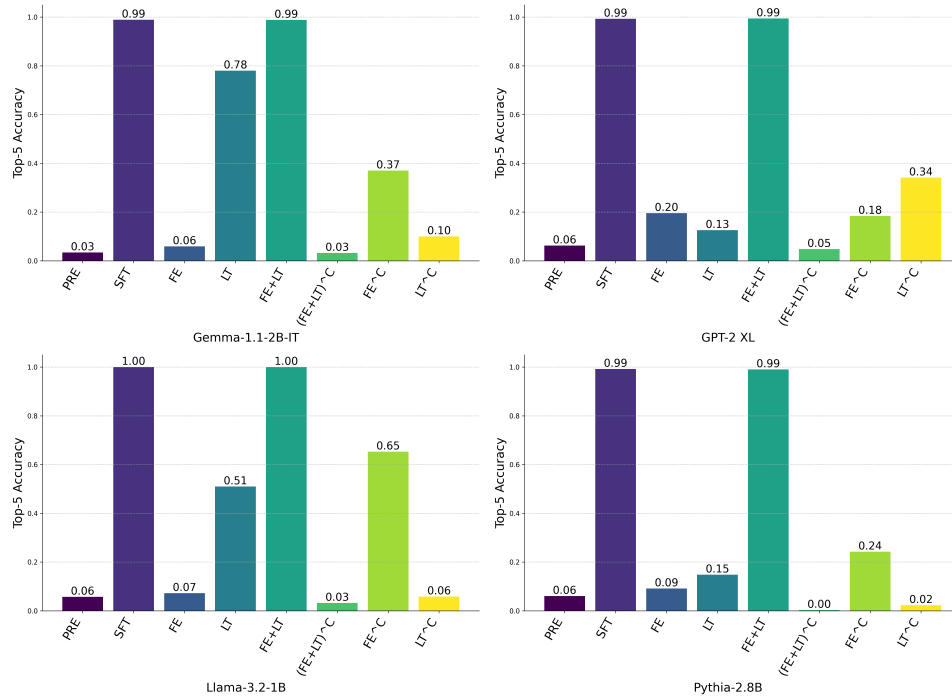


Figure 15: Top-5 accuracy — Sentence 2

C.4 ADDITIONAL RESULTS FOR REAL MOVIES, REAL ACTORS (SHUFFLED)

C.4.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

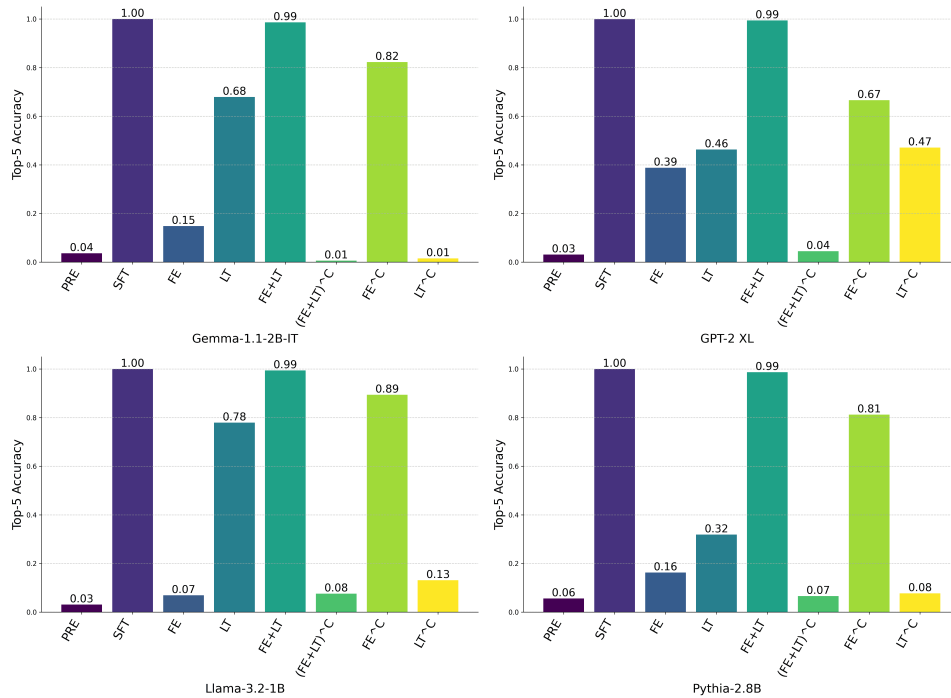


Figure 16: Top-5 accuracy — Sentence 1

C.5 MOVIE TITLE RESULTS

These experiments add the movie title to the test sentence. The movie title alone is not sufficient to recover the correct entity, but combining the movie title with the last token improves performance for all models. The movie-title-plus-first-entity condition is architecture-dependent: relative to the first entity alone, adding the movie title helps GPT-2 XL, has little effect for Llama 3 and Pythia, and hurts Gemma. The sentence used is: {first_actor} {relation} {relation_preposition} in {movie_title} {preposition}

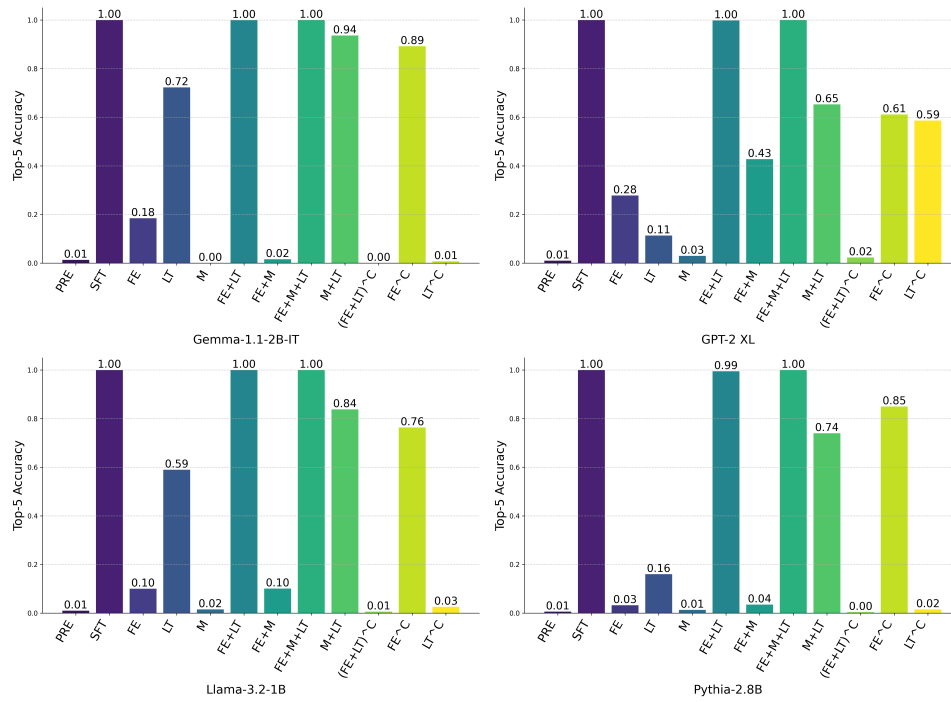


Figure 17: Top-5 accuracy–Sentence 3. “M” marks grafting at the movie-title tokens.

C.6 UNEMBEDDING MATRIX RESULTS

These experiments use finetuned unembeddings. Although top-k accuracy changes in some cases, especially for single-token-position grafts, the qualitative pattern matches the results obtained with pretrained unembeddings.

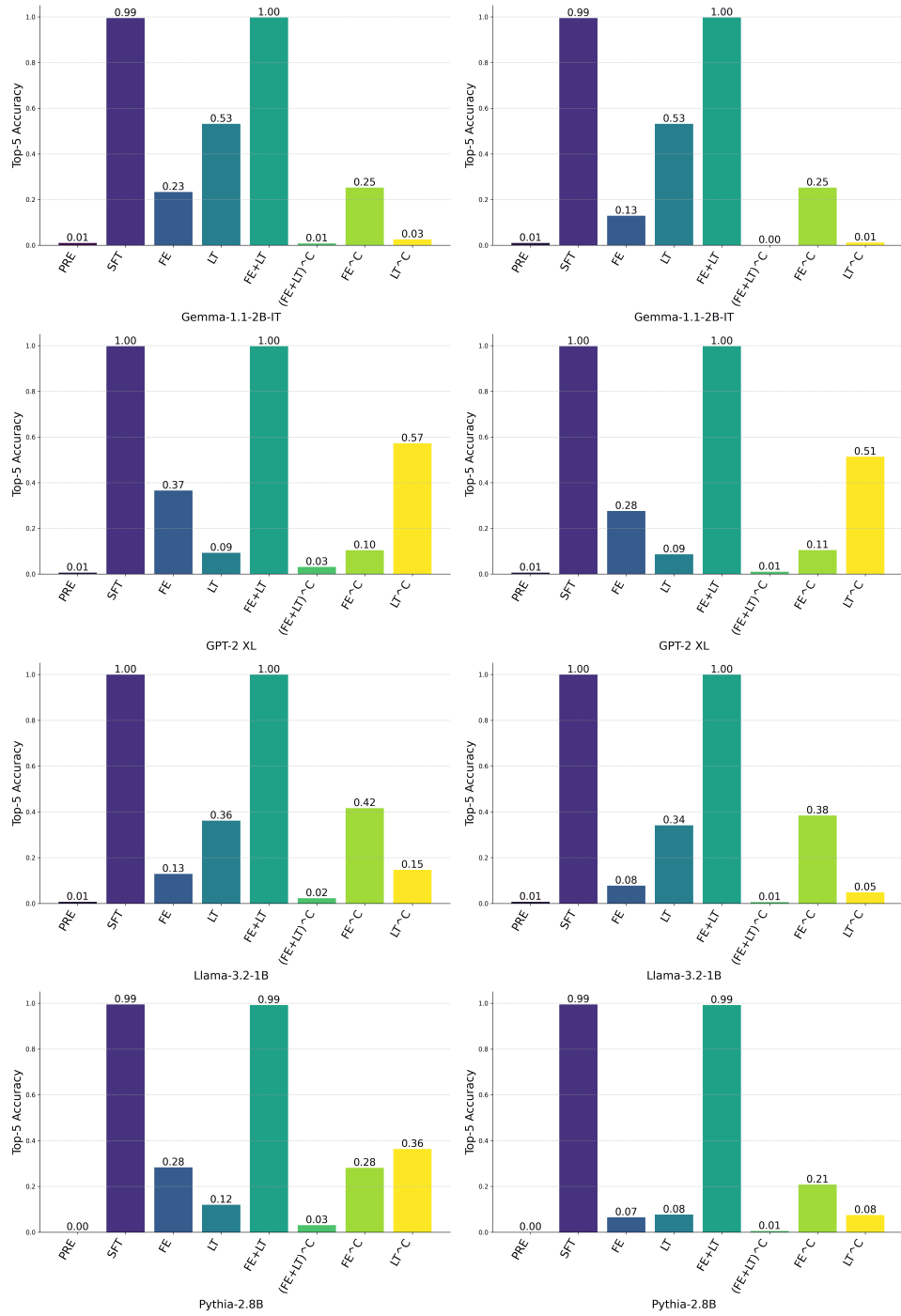


Figure 18: Top-5 accuracy–Sentence 1. Finetuned unembeddings are shown on the left and pretrained unembeddings on the right. Both choices preserve the same qualitative localization pattern, with higher top-5 accuracy for finetuned unembeddings only on the first entity.

C.7 LESS AGGRESSIVE FINETUNING RESULTS

This section reports less aggressive finetuning experiments using a lower learning rate, 0 weight decay, and supplemental openwebtext and IMDB training data. The same qualitative localization pattern remains, but the individual “extraction” and “recall” pathways are weaker.

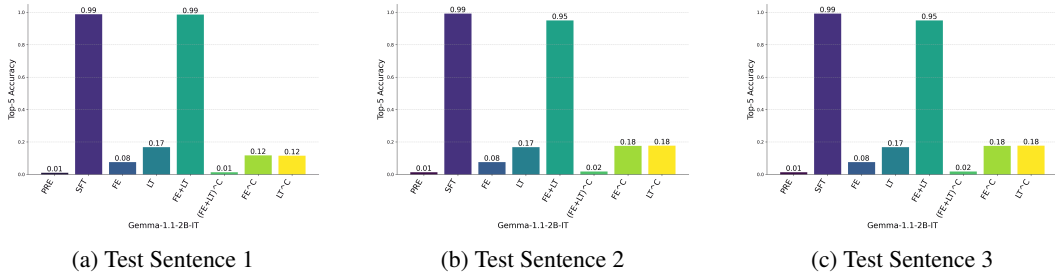


Figure 19: Top-5 accuracy for Gemma under less aggressive finetuning: lower learning rate, 0 weight decay, and supplemental openwebtext and IMDB data.

C.8 COMPONENT-GRAFTING EXPERIMENT BASELINES

This section reports baseline component-grafting results from SFT models to pretrained models.

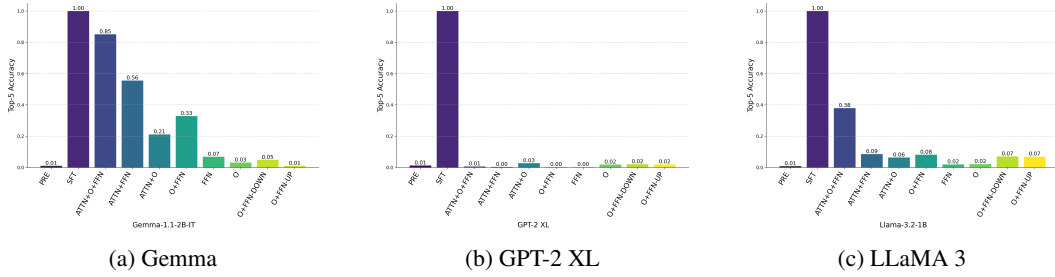


Figure 20: Top-5 accuracy for component-grafting baselines — Sentence 1

C.9 REVERSAL CURSE COMPONENT-GRAFTING EXPERIMENT RESULTS

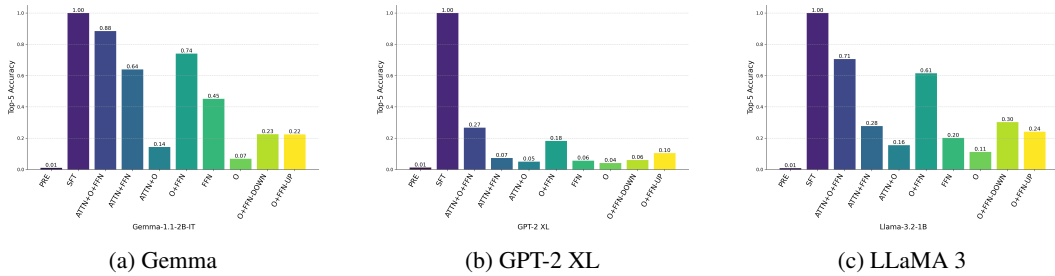


Figure 21: Top-5 accuracy for reversal-curse component-grafting experiments — Sentence 1

1566 C.10 TOKEN PROBABILITIES

1567
1568 We report five randomly sampled token-probability outputs that list the top 10 tokens for representative
1569 experimental examples. The main metric remains top-5 accuracy; these token lists provide additional
1570 context for how probability mass is distributed.
1571

1572 C.10.1 LLAMA3: FE^C

1573
1574 Example 1:
1575 Target: Carolyn: 0.002
1576 J: 0.113
1577 L: 0.041
1578 Ch: 0.040
1579 Julie: 0.034
1580 Nicole: 0.024
1581 Nick: 0.023
1582 Michael: 0.023
1583 Jon: 0.023
1584 Jamie: 0.021
1585 Kelly: 0.017

1586 -----
1587 Example 2:
1588 Target: Lindsay: 0.024
1589 Michael: 0.093
1590 Ashley: 0.061
1591 Jennifer: 0.038
1592 Kim: 0.032
1593 Alexander: 0.025
1594 Lindsay: 0.024
1595 Milo: 0.023
1596 L: 0.021
1597 Jason: 0.018
1598 Lisa: 0.018

1599 -----
1600 Example 3:
1601 Target: Gwen: 0.015
1602 Michael: 0.030
1603 Paul: 0.028
1604 Julie: 0.024
1605 Elizabeth: 0.020
1606 Mary: 0.020
1607 S: 0.019
1608 E: 0.018
1609 J: 0.017
1610 L: 0.017
1611 A: 0.017

1612 -----
1613 Example 4:
1614 Target: Dominic: 0.006
1615 Is: 0.039
1616 Sarah: 0.031
1617 Jason: 0.026
1618 D: 0.026
1619 Ellen: 0.025
Michael: 0.021
Jennifer: 0.020
Elizabeth: 0.020
Ann: 0.019

1620 Mark: 0.019

1621

1622 -----

1623

1624 Example 5:

1625 Target: Ch: 0.045

1626 Ch: 0.045

1627 David: 0.024

1628 John: 0.022

1629 Peter: 0.018

1630 L: 0.018

1631 Mark: 0.018

1632 Michael: 0.017

1633 Richard: 0.016

1634 Ben: 0.012

1635 James: 0.012

1636

1637 -----

1638

1637 C.10.2 GEMMA: LT

1638

1639 Example 1:

1640 Target: Elizabeth: 0.040

1641 Julie: 0.495

1642 John: 0.130

1643 Stephen: 0.076

1644 Elizabeth: 0.040

1645 Hilary: 0.021

1646 Laurel: 0.018

1647 Tyne: 0.017

1648 Marian: 0.014

1649 Victoria: 0.013

1650 Juliet: 0.008

1651

1652 -----

1651 Example 2:

1652 Target: Uta: 0.008

1653 John: 0.901

1654 Joan: 0.022

1655 Sally: 0.011

1656 Chelsea: 0.009

1657 Uta: 0.008

1658 Tyne: 0.003

1659 Gina: 0.002

1660 Elle: 0.001

1661 Lisa: 0.001

1662 Rosemary: 0.001

1663

1664 -----

1664 Example 3:

1665 Target: Jennifer: 0.706

1666 Jennifer: 0.706

1667 John: 0.094

1668 Il: 0.028

1669 Me: 0.020

1670 Stephen: 0.015

1671 Carol: 0.012

1672 S: 0.008

1673 David: 0.008

1674 Elle: 0.007

1675 T: 0.006

```

1674 -----
1675
1676 Example 4:
1677 Target: Marcia: 0.017
1678 John: 0.152
1679 Gwen: 0.136
1680 Susan: 0.050
1681 Tim: 0.045
1682 Ch: 0.041
1683 Celeste: 0.037
1684 Tara: 0.030
1685 T: 0.028
1686 Elle: 0.028
1687 Brittany: 0.026

```

```

1688 -----
1689 Example 5:
1690 Target: Heather: 0.035
1691 Ly: 0.916
1692 Heather: 0.035
1693 Hilary: 0.011
1694 Stephen: 0.007
1695 Julie: 0.002
1696 Rosemary: 0.002
1697 Ch: 0.002
1698 Wanda: 0.001
1699 Chelsea: 0.001
1700 Tem: 0.001
1701 -----

```

C.10.3 GPT-2 XL: (FE+LT)^C

```

1703 Example 1:
1704 Target: Fre: 0.000
1705 her: 0.037
1706 John: 0.018
1707 Jason: 0.017
1708 Chris: 0.016
1709 Adam: 0.015
1710 Zach: 0.014
1711 Josh: 0.013
1712 the: 0.013
1713 Michael: 0.013
1714 Ben: 0.012

```

```

1715 -----
1716 Example 2:
1717 Target: A: 0.002
1718 her: 0.043
1719 the: 0.017
1720 John: 0.016
1721 Tom: 0.016
1722 Peter: 0.012
1723 fellow: 0.012
1724 Michael: 0.011
1725 David: 0.011
1726 James: 0.011
1727 Jack: 0.010

```

```
1728 Example 3:
1729 Target: Matthew: 0.003
1730 his: 0.057
1731 the: 0.017
1732 Jennifer: 0.014
1733 fellow: 0.012
1734 Chris: 0.011
1735 Michael: 0.011
1736 Jason: 0.009
1737 John: 0.008
1738 James: 0.008
1739 Jessica: 0.007
```

```
1740 -----
1741 Example 4:
1742 Target: Cher: 0.000
1743 her: 0.042
1744 Tom: 0.026
1745 Robert: 0.019
1746 the: 0.018
1747 Matt: 0.018
1748 Michael: 0.016
1749 Brad: 0.016
1750 Chris: 0.014
1751 Johnny: 0.014
1752 John: 0.014
```

```
1753 -----
1754 Example 5:
1755 Target: Timothy: 0.001
1756 his: 0.056
1757 the: 0.025
1758 Michael: 0.013
1759 fellow: 0.013
1760 Robert: 0.013
1761 John: 0.012
1762 James: 0.010
1763 Tom: 0.010
1764 Mark: 0.008
1765 Peter: 0.008
1766 -----
```

1767 C.10.4 PROBABILITY ON COMMON TOKENS WITH TARGET IN TOP 5

1768 This characteristic Gemma first-entity graft shows the model assigning high probability to common
1769 tokens while still placing the target token in the top 5.
1770

```
1771 Target: Robert: 0.016
1772 a: 0.365
1773 an: 0.151
1774 the: 0.091
1775 Robert: 0.016
1776 two: 0.009
1777 his: 0.008
1778 other: 0.006
1779 John: 0.005
1780 Morgan: 0.005
1781 my: 0.005
```